

C Language By Neso Academy

"Emon SUST 01714076452"

*** U need basic of programming to read this.

Features:

1. Procedural lang (means divide gigantic program into small program called procedure or function) --> Main Feature.
2. Diff bet Low and high level lang simply points to "degree of abstraction".
 1. Low level means more effort need for user, high means less effort of user. Such as if saving a file. Only clicking save. Don't need to know what is going on background(each ans every internal details about the computer).
3. C is middle level lang, Cause it allow us to access system level feature. Such as...
 1. direct access to memory through pointer.
 2. Bit manipulation by bitwise operator.
 3. Writing assembly code within C code.

For these features c used for system level app(kernel drivers, OS).

getchar():

1. used to read a single character. This is useful in simple input-handling, like menus or character-by-character input.
2. To pause the program (e.g., "press any key to continue")
3. returns an `int`, not `char`, because it might return EOF (end of file)

#include<stdio.h> called pre-processor & header file(.h file)(which start with "#"): Replace text with actual content.

1. Replace before compilation begin.
2. Compiler convert source code into machine code.
3. <Stdio.h> header file contains declaration(on **header file**) of function like printf, scanf.

Header file Vs C std Library: C std library contains the actual definition of the function like printf, scanf.

***then why we need diff file one for storing declaration of function and one for storing definition.?

Ans(L-2, 12min): Cause there is a program called 'Linker' maps down the declaration mentioned by the preprocessor to the actual code written in the library. It simply maps, not copy paste. This linking make faster while compiling.

If all declaration and definition store on the same file. It will replace text on actual context what actually preprocessor done. Means definition will be add on source code. It increases the size of all code. It will be hazy. Computer has to do lot work to compile the program. Speed of compilation will be fall down. Therefore maintaining header file and standard library separately is essential.

Data type: simple means how much space a variable going to occupy in a memory. Requesting to compiler to allocate memory for the variable. (%d is placeholder for variable).

```

float var1 = 3.1415926535897932;
double var2 = 3.1415926535897932;
long double var3 = 3.141592653589793213456;

printf("%d\n", sizeof(float));
printf("%d\n", sizeof(double));
printf("%d\n", sizeof(long double));
printf("%.16f\n", var1);
printf("%.16f\n", var2);
printf("%.21Lf\n", var3);

```

sizeof(int) is unary operator, not function.

Modifier: short, long, signed, unsigned.

*** INT_MIN, INT_MAX(%d), UINT_MAX(%u), SHRT_MIN(%d) need <limits.h> header file.

** char declaration will be single quote('), **char a='c';**

Float -> 4 bytes = 32 bits

Double -> 8 bytes = 64 bits

Long Double -> 12 bytes = 96 bits

* Float precision upto 7 digits.

* Double precision upto 15 digits.

* Long Double precision upto 19 digits.

```

int var = 4/9;
printf("%d\n", var);

float var1 = 4/9;
printf("%.2f\n", var1);

float var2 = 4.0/9.0;
printf("%.2f\n", var2);

```



```

0
0.00
0.44

```

```

Process returned
Press any key to

```

4.0 & 9.0 will make the variable double. To make it float use **4.0f & 9.0f**.

*** Printf not only print the content on screen, but also return the number of character that it successfully printed.

** printf(“%10s”, “Hello”); will print → “ Hello” // 5 space before hello.

Global variable: variable which is declared outside of all function. Global variable by default initialize to 0.

* if we declare a global variable on a file of a project(where there are many files) then this global variable will be available to all file as global variable. If such variable we don't want to share with other file on the project then we can use static modifier. **Static int c;**

```
int var = 10;

int main() {
    int var = 3;
    printf("%d\n", var);
    fun();
    return 0;
}
```

Var inside main function will get preference over global variable.

Int var; This is declaration and definition of a variable. Declaration means here are some variable of integer type, definition means asking compiler to allocate some memory for the variable.

Auto & Extern Modifier: Variable declared inside a scope by default are automatic variable. Called auto cause it automatically destroyed after completion of function/scope. So no waste of memory.

```
#include <stdio.h>
```

```
int main() {  
    int var;  
    return 0;  
}
```

≡

```
#include <stdio.h>
```

```
int main() {  
    auto int var;  
    return 0;  
}
```

Extern modifier has only declaration, not definition. No allocation of memory. Used when a particular file needs to access that variable from another variable. This save memory and reuseability. Declaring multiple time of extern variable is allowed. Cause it is not defining.

**** Multiple definition are not allowed, like int var;**

```
int var;
```

```
int var;
```

When a extern variable is initialized, its memory is then allocated.

See Lec 13- 8.00 min

Register Modifier: Register keyword hint compiler to store variable in register memory. Simply write **Register** in front of a variable does not assure to save it on register memory. This is the choice of compiler whether it have to save on register memory or not. Most frequently used variable usually saved on register memory.

Static Modifier: if we declare a global variable on a file of a project(where there are many files) then this global variable will be available to all file as global variable. If such variable we don't want to share with other file on the project then we can use static modifier. **Static int c;**

See Lec-15, 14.00 min.

* When we declare a variable with static in local, it will initialize to 0, whereas without static modifier a variable declared, it will initialize with garbage value.

```
int main(){  
    static int a;  
    int b;  
}
```

'a' will initialize to 0, and 'b' initialize to garbage value.

Static variable retain there value between function call. **See code below**

```
1 #include<stdio.h>  
2  
3 int inc(){  
4     static int c;  
5     c++;  
6     printf("%d ", c);  
7     return c;  
8 }  
9 int main(){  
10     int v;  
11     v=inc();  
12     v=inc();  
13     v=inc();  
14     printf("%d", v);  
15     return 0;  
16 }
```

ans: 3.

Below example: here 'I' will not retain its value on every time the function called. See lec 126(pointer program problem 8)

```
void printab ()
{
    static int i, a = -3, b = -6;
    i = 0;
    while (i <= 4)
    {
        if ((i++)%2 == 1) continue;
        a = a + i;
        b = b + i;
    }
    swap (&a, &b);
    printf("a = %d, b = %d\n", a, b);
}
```

First function call: `v = inc();`

```
c
static int c; // c is initialized to 0 (only once)
c++; // c becomes 1
printf("%d ", c); // Prints "1"
return c; // Returns 1 (stored in v)
```

Memory State after 1st call:

`c = 1` (retained)

Second function call: `v = inc();`

```
c
c++; // c was 1, now becomes 2
printf("%d ", c); // Prints "2"
return c; // Returns 2 (stored in v)
```

Understanding static Variable Behavior

1. What does `static int c;` do?

- Static variables are initialized only once (to 0 if not explicitly initialized).
- They retain their value **between function calls** instead of being reinitialized each time the function runs.

In C programming, a static variable is declared using static keyword and have the property of retaining their value between multiple function calls. It is initialized only once and is not destroyed when the function returns a value. It extends the lifetime of the variable till the end of the program.

1. Static variable remains in memory even if it is declared within a block on the other hand automatic variable is destroyed after the completion of function in which it was declared.
2. Static variable if declared outside the scope of any function will act like global variable but only within the file in which it is declared.
3. You can only assign a constant literal (or value) to a static variable.

-----Static Variable Chapter Ends-----

Const: Which value cannot be modified. 2 ways to use

#define pi 3.1415 (preprocessor/macros)

const int pi=3.1415;

***Job of preprocessor(Not compiler) is to replace name with value.

#define pi 1234

int main(){

int c=3+pi; // int c=3+1234; (name replaced with value)

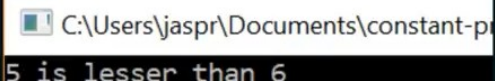
}

-----X-----

5 We can write multiple lines using \

```
#include <stdio.h>
#define greater(x, y) if(x > y) \
    printf("%d is greater than %d", x, y); \
    else \
    printf("%d is lesser than %d", x, y);

int main() {
    greater(5, 6);
    return 0;
}
```

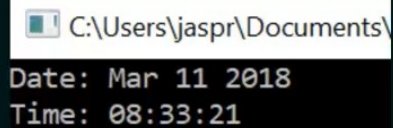


C:\Users\jaspr\Documents\constant-pr
5 is lesser than 6

7 Some predefined macros like __DATE__, __TIME__ can print current date and time.

```
#include <stdio.h>

int main() {
    printf("Date: %s\n", __DATE__);
    printf("Time: %s\n", __TIME__);
    return 0;
}
```



C:\Users\jaspr\Documents\
Date: Mar 11 2018
Time: 08:33:21

***Why Need '&' on scanf?

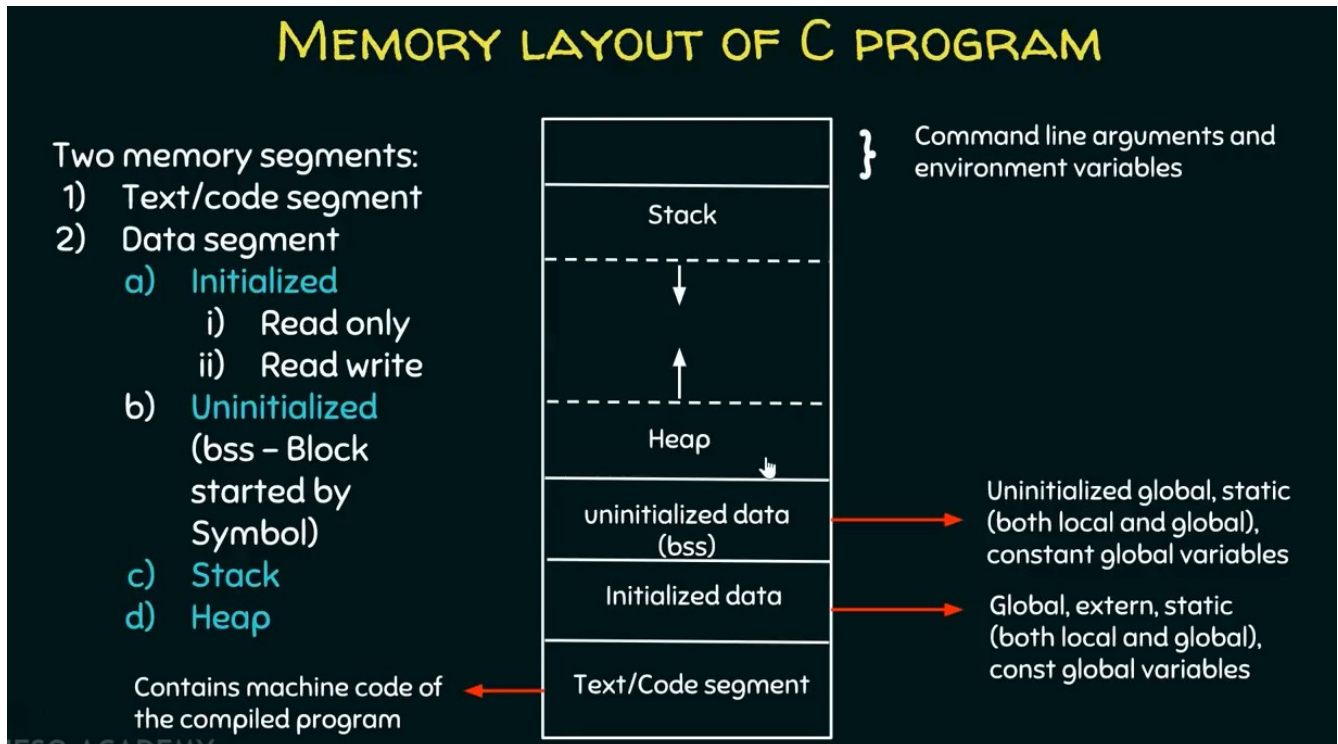
While scanning the input, scanf needs to store that input data somewhere.

To store this input data, scanf needs to know the memory location of a variable.

```
int a=053; // represent octal number  
int a=0x43aa; // represent hexadecimal number
```

-----X-----

Memory Layout:



See **Lec-20** for memory mapping understanding.

Stack: (Lec-69(Static & Dynamic scoping))

* Stack is container/ memory segment which holds some data. Data is retrieved in it in LIFO fashion.

Two Operation- **PUSH, POP.**

Whenever we call a function it will store on **Stack**. In reality it is not the function which is stored in stack. It is the "**Activation Record**" which is maintained inside the Called stack/stack.

Activation Record:

1. Local variable of the function.
2. Return address means- where it need to go back when this function is executed. In below example when 'fun1' function execution completed it return back to "a=fun1(a);" of main function through return address.

Activation Record () – is a portion of a stack which is generally composed of:

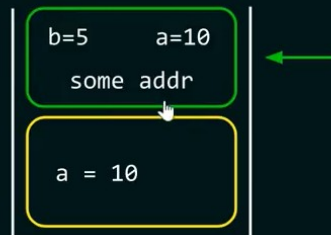
1. Locals of the callee
2. Return address to the caller
3. Parameters of the callee

Example:

```
int main()
{
    int a = 10;
    a = fun1(a);
    printf("%d", a);
}

int fun1(int a)
{
    int b = 5;
    b = b+a;
    return b;
}
```

```
fun1() { fun2(); }
fun2() { fun3(); }
fun3() { return; }
```



STACK (or
CALL STACK)



STACK (or
CALL STACK)

```

1 #include<stdio.h>
2
3 int a=34;
4 a=23;
5 int main(){
6     printf("%d", a);
7 }

```

```
$ gcc prog.c -Wall -Wextra -std=gnu11
```

Run

 Share

```

prog.c:4:1: warning: data definition has no type or storage class
  4 | a=23;
    | ^
prog.c:4:1: warning: type defaults to 'int' in declaration of 'a' [-W
prog.c:4:1: error: redefinition of 'a'
prog.c:3:5: note: previous definition of 'a' with type 'int'
  3 | int a=34;
    |     ^

```

Assigning a value in global variable is not allowed. Compiler automatically set 'int' before 'a' while assigning. That's why redefinition says. Lec-20

** Assignment is allowed on function/on a scope block only, Not globally

-----XX-----

Operator:

Name of operators	Operators
Arithmetic operators	+ , - , * , / , %
Increment/Decrement operators	++, --
Relational operators	== , != , < , > , <= , >= , < , >

Name of operators	Operators
Logical operators	&& , , !
Bitwise operators	& , ^ , , ~ , >> , <<
Assignment operators	= , += , -= , *= , /= , %= , <<= , >>= , &= , ^= , =

-----X-----

a++, ++a:

Pre – means first increment/decrement then assign it to another variable .

Post – means first assign it to another variable then increment/decrement. 

Below code output is “5 7”

```
1 #include<stdio.h>
2
3 int main(){
4     int x=5;
5     int xx=x++;
6     int xxx=++x;
7     printf("%d %d", xx,xxx);
8 }
```

-----X-----

Token Generation:

- ★ Lexical analysis is the first phase in the compilation process.
- ★ Lexical analyzer (scanner) scans the whole source program and when it finds the meaningful sequence of characters (lexemes) then it converts it into a token
- ★ **Token:** lexemes mapped into token-name and attribute-value.
Example: int → <keyword,int>

In C, tokens are the **smallest meaningful units** of code. The compiler breaks code into these tokens during lexical analysis.

♦ Types of Tokens:

1. **Keywords** → int, if, return
2. **Identifiers** → myVariable, func1
3. **Operators** → +, =, *
4. **Literals** → "hello", 123
5. **Punctuation/Special symbols** → {, ;, ()
6. **Preprocessor directives** → #define, #include

```
int x = 10 + 5;
```

• Tokens Generated:

- `int` (Keyword)
- `x` (Identifier)
- `=` (Operator)
- `10` (Integer Literal)
- `+` (Operator)
- `5` (Integer Literal)
- `;` (Punctuation)

-----XXXX-----

```
#include <stdio.h>
```

```
int main() {  
    int a = 4, b = 3;  
    printf("%d", a + ++b);  
    return 0;  
}
```

Output:

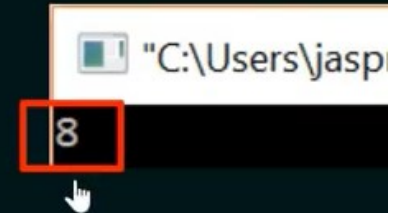
 "C:\Users\jaspi

8

```
#include <stdio.h>
```

```
int main() {  
    int a = 4, b = 3;  
    printf("%d", a + ++b);  
    return 0;  
}
```

Output:



See Lec-24

-----X-----



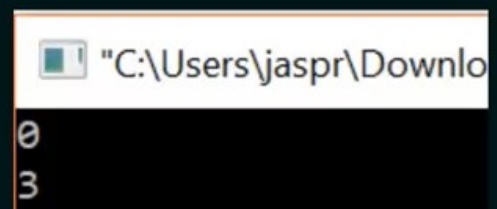
All Relational operators will return either True or False.

-----X-----

short circuit:

```
#include <stdio.h>
```

```
int main() {  
    int a = 5, b = 3;  
    int incr;  
  
    incr = (a < b) && (b++);  
    printf("%d\n", incr);  
    printf("%d", b);  
    return 0;  
}
```

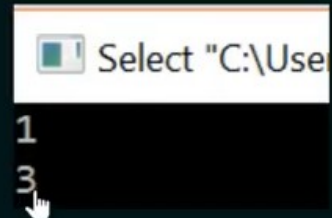


***Here in and(&&) logical operator if 1st condition is false 2nd condition will not evaluated or checked.


```
#include <stdio.h>

int main() {
    int a = 5, b = 3;
    int incr;

    incr = (a > b) || (b++);
    printf("%d\n", incr);
    printf("%d", b);
    return 0;
}
```



***Here in or(||) logical operator if 1st condition is true 2nd condition will not evaluated or checked.
 *** Both called “Concept of **short circuit** in logical operators”

-----X-----

Bitwise operator:

Bitwise NOT(~) operator is unary operator. Means it need 1 operand.

BITWISE NOT (~) OPERATOR

- NOT is a unary operator
- It's job is to complement each bit one by one.
- Result of NOT is 0 when bit is 1 and 1 when bit is 0

7 → ~ 0 1 1 1
 8 ← 1 0 0 0

$$\sim 7 = 8$$

Truth Table

A	~A
0	1
1	0

Here Complement means 1 become 0 and 0 becomes 1.

Left Shift:

② Left shifting is equivalent to multiplication by $2^{\text{rightOperand}}$

Example:

```
var = 3;
```

```
var << 1
```

Output: 6 [3 x 2¹]

Right Shift:

② Right shifting is equivalent to division by $2^{\text{rightOperand}}$

Example:

```
var = 3;
```

```
var >> 1
```

Output: 1 [3 / 2¹]

```
var = 32;
```

```
var >> 4
```

Output: 2 [32 / 2⁴]

XOR:

Exclusive OR

- Either A is 1 or B is 1 then the output is 1 but when both A and B are 1 then output is 0.
- Excluding BOTH

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

Comma Operator:

 Comma operator returns the rightmost operand in the expression and it simply evaluates the rest of the operands and finally reject them.

Example:

```
int var = (printf("%s\n", "HELLO!"), 5);  
printf("%d", var);
```

Output: HELLO!
5

This value will be returned to var after evaluating the first operand

It will simply not rejected. First evaluated and then rejected.

Or

```

1 #include <stdio.h>
2
3 int main(){
4     int a=65;
5     int var=(3,3,a++,2);
6     printf("%d %d", a,var);
7
8 }

```

Result will be – a=55, var=2;

Many things to know about comma operator.

For more see lec-33(Comma operator)

-----X-----

Example:

```

int main()
{
    int a;
    a = fun1() + fun2();
    printf("%d", a);
    return 0;
}

```

```

int fun1()
{
    printf("Neso");
    return 1;
}

```

```

int fun2()
{
    printf("Academy");
    return 1;
}

```

Which function is called first ?
fun1() or fun2()?

It is not defined whether fun1() will be called first or whether fun2() will be called. Behaviour is undefined and output is compiler dependent.

C99 is the older version of C standard adopted in 1999.
C11 is the latest revised version of C standard adopted in 2011. 🖱️

C Standard: C Standard is the language specification(Rules, Syntax, Feature etc) which is adopted by the all C compiler across the globe. Such as, about the “FOR” loop, something is written inside the standard that explain its functionality, then all C compiler must have to adopt the functionality of “FOR” loop which is provided by C standard.

QUIZ:

```
#include <stdio.h>
int main()
{
    int i = 5;
    int var = sizeof(i++);
    printf("%d %d", i, var);
    return 0;
}
```

Ans: i=5, var=4;

Here 'i++' will not be evaluated. Because 'sizeof' is a compile-time operator. It only determines the size of the type. See **Lec-35**(Operator in C, solved problem 1);

◆ Understanding sizeof(i++)

1. sizeof is a compile-time operator

- It does **not** evaluate the expression inside it.
- It only determines the size of the type of the expression.

2. Does i++ increment i?

- **No.** Since sizeof(i++) only checks the type of i, the ++ operation is **not performed**.
- i remains 5 (unchanged).

-----X-----

****Post-increment** Means after the completion of the expression increment will be evaluated.

```
1 #include <stdio.h>
2 int main() {
3     int c=9;
4     printf("%d", c++);
5     return 0;
6 }
```

Here ans will c=9;

after expression completed c will increment to 10.

QUIZ:

Q8: Predict the output

```
# include <stdio.h>
int var = 5;
int main() {
    int var = var;
    printf("%d", var);
}
```

Ans will be **garbage value**. Because local and global variable has same name, so local variable is shadowing global variable. Wrong Initialization.

-----X-----

Switch: Switch is great replacement of long **if-else** construct.

***Default** case is optional in switch statement.

ALLOWED

```
int main() {
    int a = 1, b = 2, c = 3;
    switch(a+b*c)
    {
        case 1: printf("choice 1");
                break;
        case 2: printf("choice 2");
                break;
        default: printf("default");
                break;
    }
}
```

default

NOT ALLOWED

```
int main() {
    float a = 1.15, b = 2.0, c = 3.0;
    switch(a+b*c)
    {
        case 1: printf("choice 1");
                break;
        case 2: printf("choice 2");
                break;
        default: printf("default");
                break;
    }
}
```

```
prog.c: In function 'main':
prog.c:5:9: error: switch quantity not an integer
switch(a+b*c)
      ^
```


NOT ALLOWED

```
int main() {
    float x = 3.14;
    switch(x)
    {
        case 3.14: printf("x is 3.14");
                    break;
        case 1.1: printf("x is 1.14");
                   break;
        case 2: printf("x is 2");
                 break;
    }
}
```

```
prog.c:7:6: error: case label does not reduce to an integer constant
case 3.14: printf("x is 3.14");
   ^
prog.c:9:6: error: case label does not reduce to an integer constant
case 1.1: printf("x is 1.14");
   ^
```

ALLOWED

```
int main() {
    int x = 23;
    switch(x)
    {
        case 3+3: printf("choice 1");
                   break;
        case 3+4*5: printf("choice 2");
                     break;
        default: printf("default");
                  break;
    }
}
```

choice 2

4

Variable expressions are not allowed in case labels. **Although macros are allowed.**

```
#include <stdio.h>
#define y 2
#define z 23

int main() {
    int x = 2;
    switch(x)
    {
        case y: printf("Number is 2");
                 break;
        case z: printf("Number is 23");
                 break;
        default: printf("default case");
                  break;
    }
}
```

Output:

Number is 2

5

Default can be placed anywhere inside switch. It will still get evaluated if no match is found.

```
int main() {
    int x = 2, y = 2, z = 23;
    switch(x)
    {
        case y: printf("Number is 2");
                break;
        case z: printf("Number is 23");
                break;
        default: printf("default case");
                break;
    }
}
```

```
prog.c:7:6: error: case label does not reduce to an integer constant
    case y: printf("Number is 2");
    ^
prog.c:9:6: error: case label does not reduce to an integer constant
    case z: printf("Number is 23");
```

CADEMY

```
int main(){
    int i=0;
    for(i=0; i<20; i++){
        switch(i){
            case 0: i+=5;
            case 1: i+=2;
            case 5: i+=5;
            default: i+=4;
        }
        cout<<i<<" ";
    }
}
```

* If **“Break”** statement is not used, then if one statement is matched, all subsequent cases will be executed until end of switch case. Here, on 1st loop case-0,1,5,default will execute, on 2nd loop only default will execute.

-----X-----

Binary To Decimal:

```
printf("Enter the binary number: ");
scanf("%d", &binary);

decimal = 0, weight = 1;

while(binary != 0)
{
    rem = binary % 10;
    decimal = decimal + rem*weight;
    binary = binary / 10;
    weight = weight*2;
}
```

1 0 0 1

rem = 1

decimal = 1

binary = 1 0 0

weight = $1 \times 2 = 2$

Power of an Int: For Negative Power(2^{-3}). For Positive exponent find what to change.

IMPLEMENTATION

```
int base, exponent, expo;
double power1=1.0;
printf("Enter the base: ");
scanf("%d", &base);
printf("Enter the exponent: ");
scanf("%d", &exponent);

expo = exponent;
if(exponent < 0)
{
    while(exponent!=0)
    {
        power1 = power1*(1.0/base);
        exponent++;
    }
    printf("%d to the power of %d is %.10f", base, expo, power1);
}
```

exponent
-3
2
base

Iteration 1:
 $\text{power1} = 1 \times (1.0/2) = 0.5$
 $\text{exponent} = -3+1 = -2$

Function: In C, Functions are global by default.

WHAT IS THE DIFFERENCE BETWEEN ARGUMENTS AND PARAMETERS?

```
int add(int, int);

int main()
{
    int m = 20, n = 30, sum;
    sum = add(m, n);
    printf("sum is %d", sum);
}

int add(int a, int b)
{
    return (a + b);
}
```

Arguments or Actual Parameters

Parameters or Formal Parameters

Call By Value:

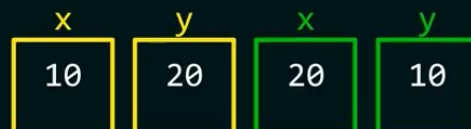
CALL BY VALUE

Here values of actual parameters will be copied to formal parameters and these two different parameters store values in different locations

```
int x = 10, y = 20;
fun(x, y);
printf("x = %d, y = %d", x, y);

int fun(int x, int y)
{
    x = 20;
    y = 10;
}
```

Output: x = 10, y = 20



Call By reference:

CALL BY REFERENCE

Here both actual and formal parameters refers to same memory location. Therefore, any changes made to the formal parameters will get reflected to actual parameters

Here instead of passing values, we pass addresses

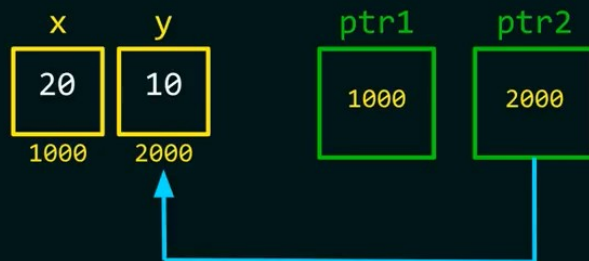
*** &x and *ptr here '&(reference Operator)' referencing value, '*'(De-reference Operator)' is de-referencing value. They are opposite to each other.

CALL BY REFERENCE

```
int x = 10, y = 20;  
fun(&x, &y);  
printf("x = %d, y = %d", x, y);
```

```
int fun(int *ptr1, int *ptr2)  
{  
    *ptr1 = 20;  
    *ptr2 = 10;  
}
```

Output: x = 20, y = 10



-----X-----

Static Function: Static functions are restricted to their file where they are declared. No other file can't access these functions.

In C, functions are global by default.

This means if we want to access the function outside from the file where it is declared, we can access it easily.

Now if we want to restrict this access, then we make our function static by simply putting a keyword **static** in front of the function.

See Lec-68(Static Function) for details understanding

-----X-----

Static & Dynamic Scoping: C lang follows static scoping. To know details about Dynamic scoping See Lec-69-71(Static & Dynamic scoping).

If a variable is not found so that scope, then it will search outer containing block or scope. Like

```
int a=5;
|
int main(){
    int a=10;
    if(1){
        int b=10;
        b=b+a;
    }
}
```

Here in 'if' statement a=10 will count. If a=10 if comment out then a=5 will count.

Below Example...

```
int fun1(int);
int fun2(int);
int a = 5;
int main()
{
    int a = 10;
    a = fun1(a);
    printf("%d", a);
}

int fun1(int b)
{
    b = b+10;
    b = fun2(b);
    return b;
}

int fun2(int b)
{
    int c;
    c = a + b;
    return c;
}
```

Output: 25

This ans is for Static Scoping.

Here in 'fun2' 'a' variable is not found on this scope. It will search for 'a' outer scope. So a=5;

But in case of **Dynamic scoping**

In dynamic scoping, definition of a variable is resolved by searching its containing block and if not found, then searching its calling function and if still not found then the function which called that calling function will be searched and so on.

DYNAMIC SCOPING EXAMPLE

```
int fun1(int);
int fun2(int);
int a = 5;
int main()
{
    int a = 10;
    a = fun1(a);
    printf("%d", a);
}

int fun1(int b)
{
    b = b+10;
    b = fun2(b);
    return b;
}

int fun2(int b)
{
    int c;
    c = a + b;
    return c;
}
```

Output: 30

In 'fun2' 'a' variable is not found, so it will search on callee function(fun1) for 'a'. if not found it search again for that callee function(main) and use a=10; if not found use global variable a=5;

Thats why for static scoping ans=25;

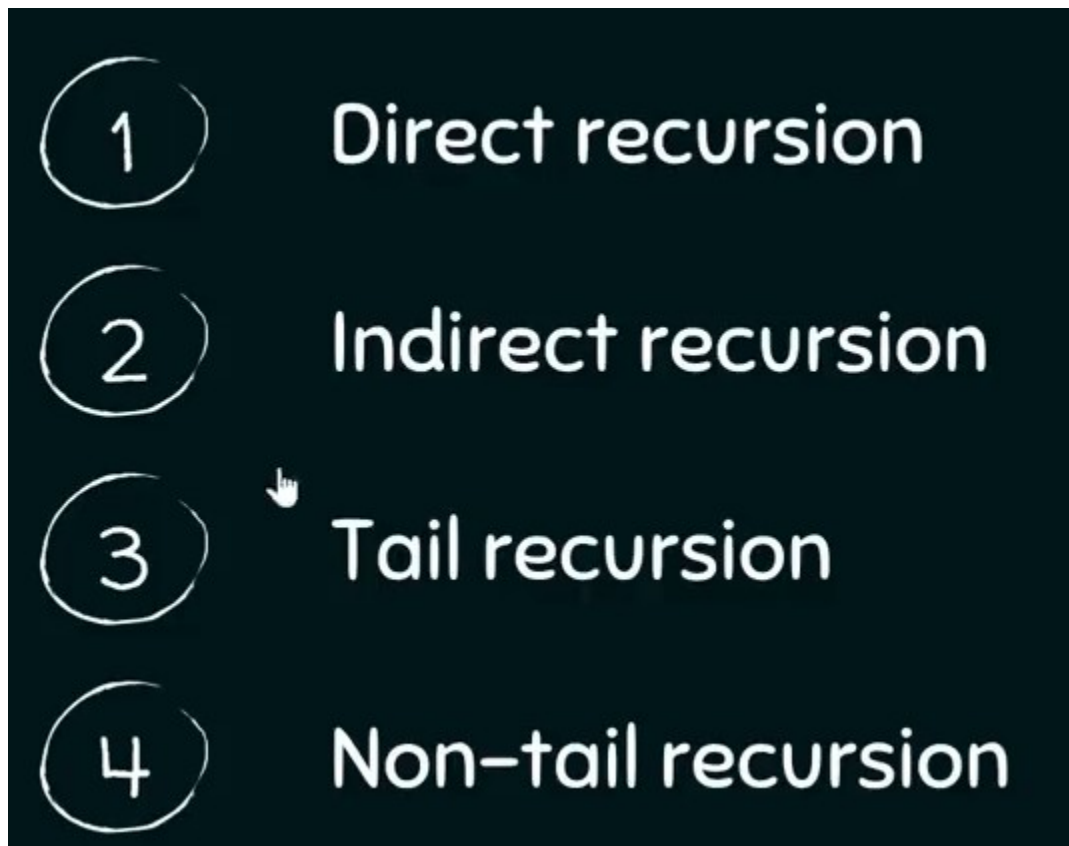
for Dynamic scoping ans=30;

-----X-----

Recursion:

* Must specify base condition to stop the recursion.

Type of Recursion:



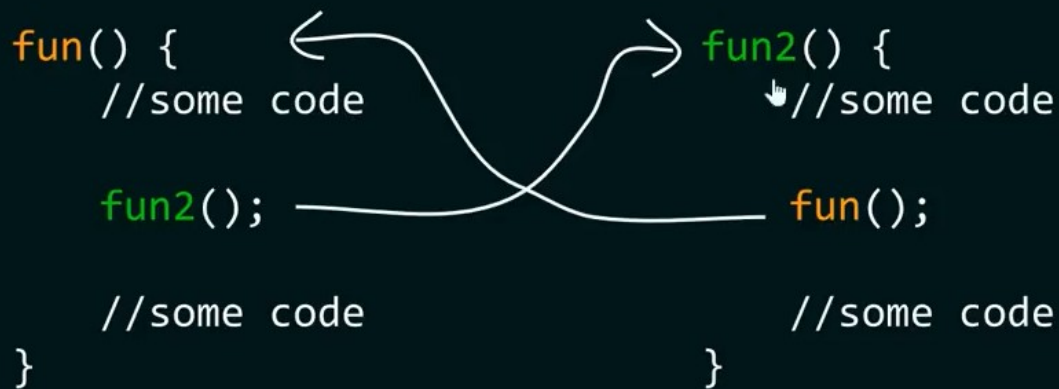
Direct recursion

A function is called direct recursive if it calls the same function again.

Indirect recursion

A function (let say **fun**) is called indirect recursive if it calls another function (let say **fun2**) and then **fun2** calls **fun** directly or indirectly.

Structure of Indirect recursion:



A recursive function is said to be **tail recursive** if the recursive call is the last thing done by the function. There is no need to keep record of the previous state.

```
void fun(int n) {  
    if(n == 0)  
        return;  
    else  
        printf("%d ", n);  
    return fun(n-1);  
}  
int main() {  
    fun(3);  
    return 0;  
}
```



Output: 3 2 1

A recursive function is said to be **non-tail recursive** if the recursive call is not the last thing done by the function. After returning back, there is some something left to evaluate.

```
void fun(int n) {  
    if(n == 0)  
        return;  
    fun(n-1);  
    printf("%d ", n);  
}  
int main() {  
    fun(3);  
    return 0;  
}
```

Output: 1 2 3

Advantage and Disadvantage: Every Recursive program can be written as iterative program.

- * Recursive programs are more elegant and requires less lines of code

- * Recursive program requires more space(Stack Space) than iterative programs. Because in recursion for every function call there is a activation record stored on call stack. But no function are stored on iterative..

Const Array: Can't change ant element of an array. Const a[3]={1,2,3};

Pointer

Pointer is a special variable which is capable of storing the initial address of an object which it wants to point. Simply point to the base address of the object. Variable **int i=6**; address={101, 102, 103, 104}. pointer will point(store address) to 101 address.

int x; // declaring variable.

X=5; // initializing variable

int *ptr=&x; // pointer **ptr** is pointing to variable **x**.

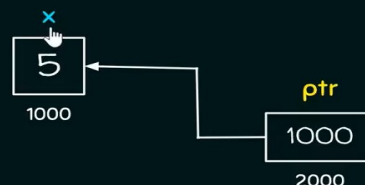
Dereference Operator:

Value of operator/indirection operator/dereference operator is an operator that is used to access the value stored at the location pointed by the pointer.

```
int x = 5;  
int *ptr;  
ptr = &x;  
printf("%d", *ptr);
```

It says go to the address of object and take what is stored in the object

VALUE OF OPERATOR



Can change variable value with pointer.

Int x=5, *ptr=&x;

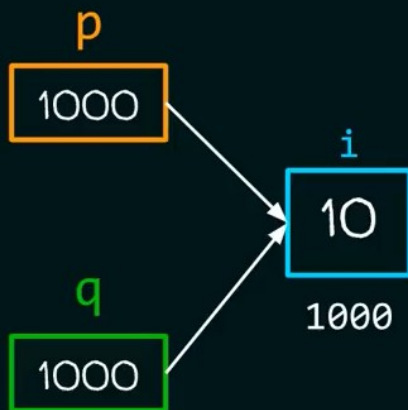
*prt=4; // x value will be 4.

```
int x=6;  
int *ptr;  
ptr=&x;  
*ptr=5; // while changing variable value '*' have to use  
printf("%d", *ptr);
```

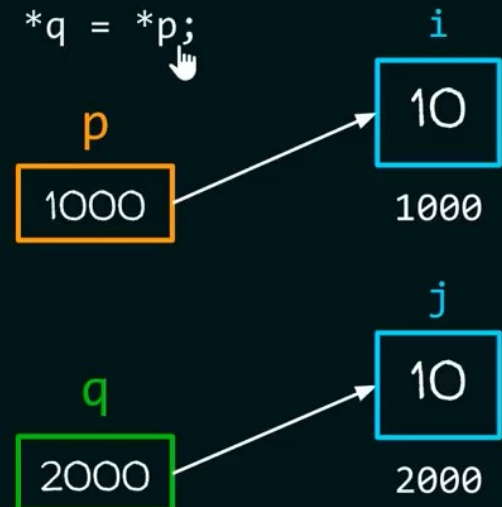
*Important

NOTE: $q = p$ is not same as $*q = *p$

```
int i = 10;  
int *p, *q;  
p = &i;  
q = p;
```



```
int i = 10, j = 20;  
int *p, *q;  
p = &i;  
q = &j;  
*q = *p;
```



Here value changes to 10 from 20.

****Important:**

```
#include <stdio.h>

void minMax(int arr[], int len, int *min, int *max)
{
    *min = *max = arr[0];
    int i;
    for(i=1; i<len; i++)
    {
        if(arr[i] > *max)
            *max = arr[i];
        if(arr[i] < *min)
            *min = arr[i];
    }
}

int main() {
    int a[] = {23, 4, 21, 98, 987, 45, 32, 10, 123, 986, 50, 3, 4, 5};
    int min, max;
    int len = sizeof(a)/sizeof(a[0]);
    minMax(a, len, &min, &max);
    printf("Minimum value in the array is: %d and Maximum value is: %d", min, max);
    return 0;
}
```

See "Lec:- 106-Pointer application"

Return pointer(address): Finding array mid value by pointer

```
int main()
{
    int a[] = {1, 2, 3, 4, 5};
    int n = sizeof(a)/sizeof(a[0]);
    int *mid = findMid(a, n);
    printf("%d", *mid);
    return 0;
}

int *findMid(int a[], int n)
{
    return &a[n/2];
}
```


****important:** automatic variable means local variable

Never ever try to return the address of an **automatic variable**

For example:

```
int *fun()
{
    int i=10;
    return &i;
}

int main() {
    int *p = fun();
    printf("%d", *p);
}
```

Warning: function returns address of local variable

Question 1: Consider the following two statements

```
int *p = &i;
p = &i;
```

First statement is the declaration and second is simple assignment statement. Why isn't in second statement, p is preceded by * symbol?

Solution: In C, * symbol has different meanings depending on the context in which it's used.

At the time of declaration, * symbol is not acting as an indirection operator.

* symbol in the first statement tells the compiler that p is a pointer to an integer.

But, if we write *p = &i then it is wrong, because here * symbol indicates the indirection operator and we cannot assign the address to some integer variable.

Therefore, in the second statement, there is no need of * symbol in front of p. It simply means we are assigning the address to a pointer.

****Printing address of a pointer. `Printf("%p", ptr);`**

Question 4: If *i* is a variable and *p* points to *i*, which of the following expressions are aliases of *i*?

- a) **p* b) **&p* c) *&p* d) **i* e) **&i*

Solution: Options (a) and (e) are the aliases of variable *i*

Here “**&i*” “**p*” gives the same value.

Question 4: If *i* is a variable and *p* points to *i*, which of the following expressions are aliases of *i*?

- a) **p* b) **&p* c) *&p* d) **i* e) **&i*

Solution: Options (a) and (e) are the aliases of variable *i*

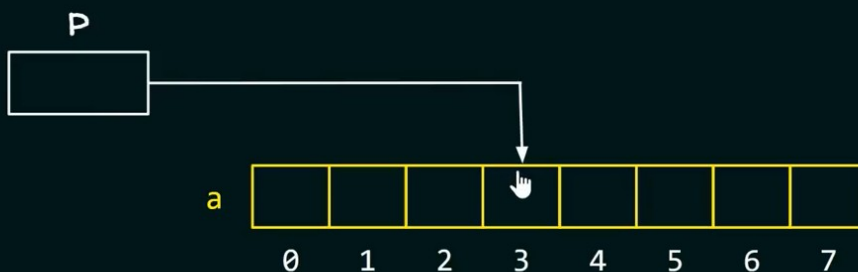
Example

```
int i = 10;  
int *p = &i;
```



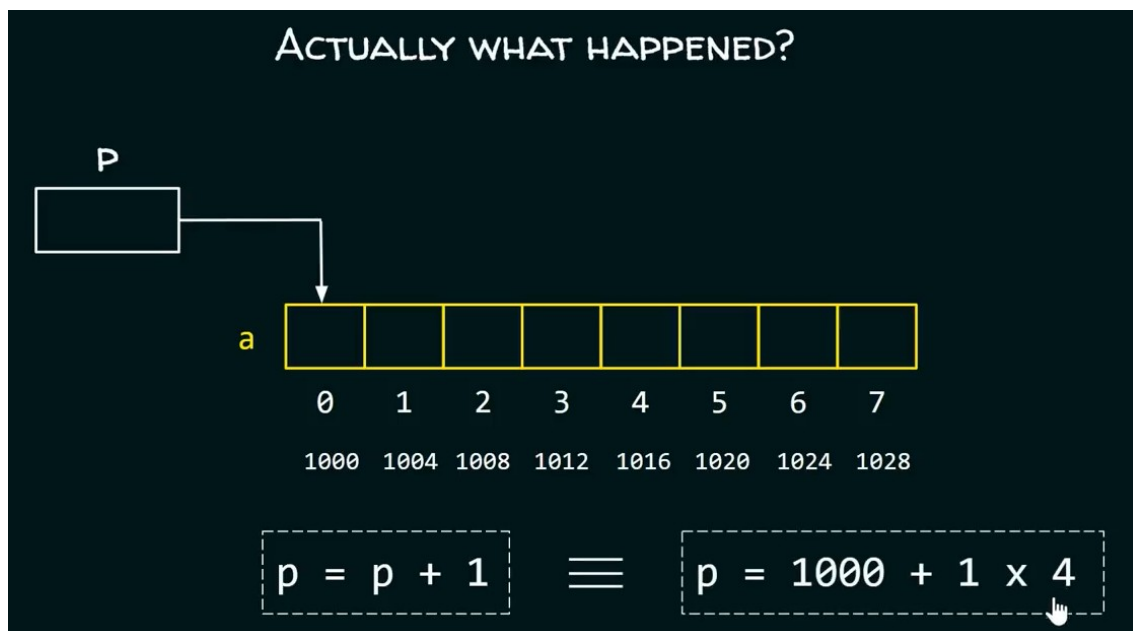
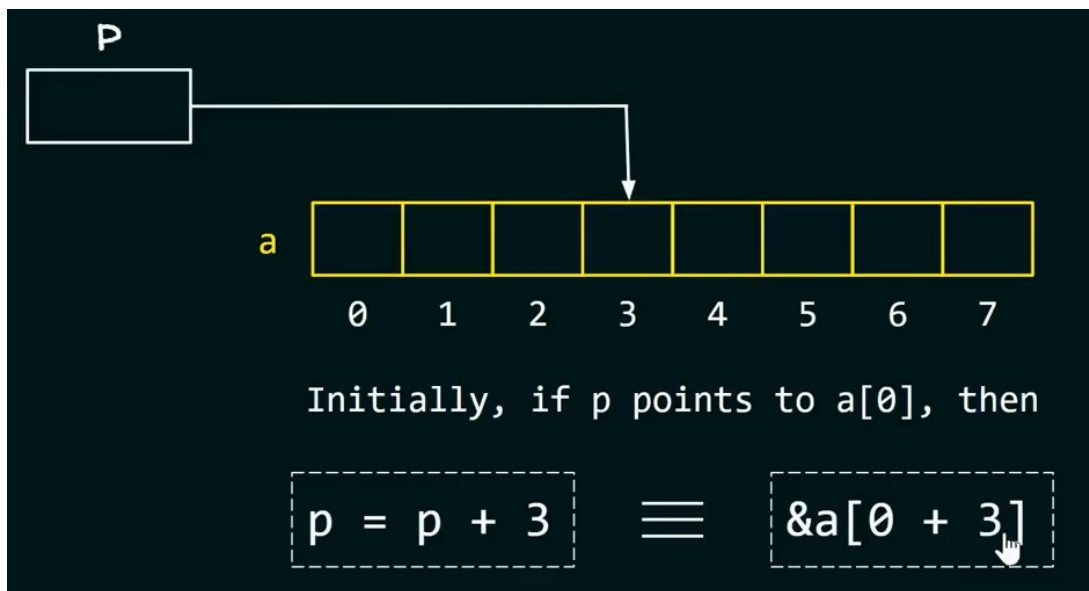
- a) **p* = **(1000)* = 10
b) **&p* = **(&p)* = **(2000)* = 1000
c) *&p* = 2000
d) **i* = **(10)* doesn't make sense
e) **&i* = **(&i)* = **(1000)* = 10

WHAT HAPPENS IF WE ADD SOME INTEGER TO THE POINTER?



$p = p + 3$

means moving the pointer 3 positions in forward direction.



***Same rule for subtraction.

```

int main() {
    int a[] = {5, 16, 7, 89, 45, 32, 23, 10};
    int *p = &a[0];
    printf("%d ", *(p++));
    printf("%d", *p);
    return 0;
}

```

Ans: 5 16

*** Compare the results...

```

int a[8]={5,16,7,89,45,32,23,10};
int *p=&a[1], *q=&a[5];
printf("%d ", *(p++)); //ans: 16
printf("%d\n", *(q++)); //ans: 32
printf("%d ", *(p+3)); //ans: 32
printf("%d\n", *(q-3)); //ans: 89
printf("%d ", *p); //ans: 7
printf("%d\n", *q); //ans: 23
printf("%d ", q-p); //ans: 4
printf("%d ", p<q); //ans: 1
printf("%d ", *p<*q); //ans: 1

```

Remember: If using “++” pointer will change its location, but using “+ or -” does not changes location.

FACT: NAME OF AN ARRAY CAN BE USED AS A POINTER TO THE **FIRST ELEMENT** OF AN ARRAY.

For example:

```

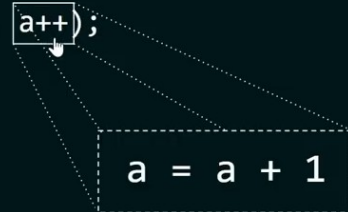
int main() {
    int a[5];
    *a = 10;
    printf("%d", a[0]);
    return 0;
}

```

It is true that we can use array names as pointers, but assigning a new address to them is not possible

For example:

```
int main() {  
    int a[] = {11, 22, 36, 5, 2};  
  
    printf("%p", a++);  
    return 0;  
}
```



a	11	22	36	5	2
	1000	1004	1008	1012	1016

We are trying to assign address 1004 to array name 'a'

Recall that name of the array indicates the base address of the array i.e. 1000. We cannot change this.

This is not allowed. Because we are changing the base address from 1000 to 1004. Instead of this we can write below code.

There is no problem in the code below.

For example:

```
int main() {  
    int a[] = {11, 22, 36, 5, 2};  
  
    printf("%p", a+1);  
    return 0;  
}
```

a	11	22	36	5	2
	1000	1004	1008	1012	1016

Here, we are not trying to assign some new address to 'a'.

We are simply accessing the address of the second element of the array.

UNDERSTAND THE DIFFERENCE BETWEEN
ACCESSING AND ASSIGNING.

Alternative: Assigning the base address to new pointer.

```
int main() {  
    int a[] = {11, 22, 36, 5, 2};  
    int *p = a;  
    printf("%d", *(++p));  
    return 0;  
}
```

OUTPUT: 22

-----X-----

OLD PROGRAM

```
int main() {  
    int a[] = {11, 22, 36, 5, 2};  
    int sum = 0, *p;  
  
    for(p = &a[0]; p <= &a[4]; p++)  
        sum += *p;  
  
    printf("Sum is %d", sum);  
}
```

Output: Sum is 76

NEW PROGRAM

```
int main() {  
    int a[] = {11, 22, 36, 5, 2};  
    int sum = 0, *p;  
  
    for(p = a; p <= a + 4; p++)  
        sum += *p;  
  
    printf("Sum is %d", sum);  
}
```

Output: Sum is 76

*Print array reverse order by pointer:

```
#include <stdio.h>
#define N 5

int main() {
    int a[N], *p;
    printf("Enter 5 elements in the array: ", N);
    for(p=a; p <= a+N-1; p++)
        scanf("%d", p);

    printf("Elements in reverse order:\n");
    for(p = a+N-1; p >= a; p--)
        printf("%d ", *p);

    return 0;
}
```



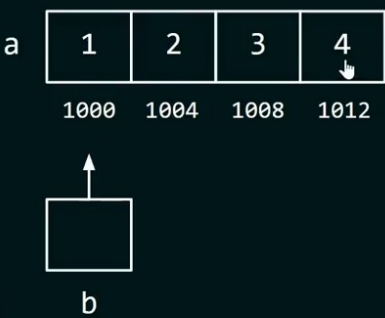
*while passing an array to a function we actually pass the base address to the function.

SIMPLE EXAMPLE

```
int add(int b[], int len)
{
    int sum = 0, i;
    for(i=0; i<len; i++)
        sum += b[i];
    return sum;
}

int main() {
    int a[] = {1, 2, 3, 4};
    int len = sizeof(a)/sizeof(a[0]);
    printf("%d", add(a, len));
    return 0;
}
```

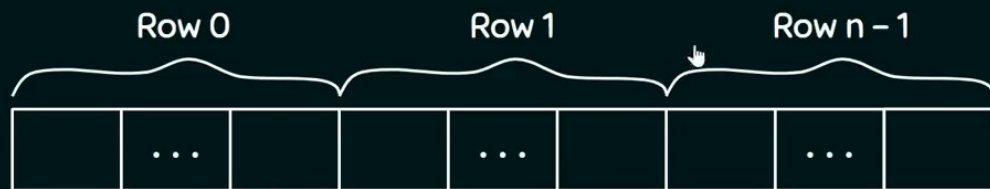
You can also write `int *b;`



Row/ Column major order:

DIFFERENCE BETWEEN ROW MAJOR AND COLUMN MAJOR ORDER

Row major order: Elements are stored row by row

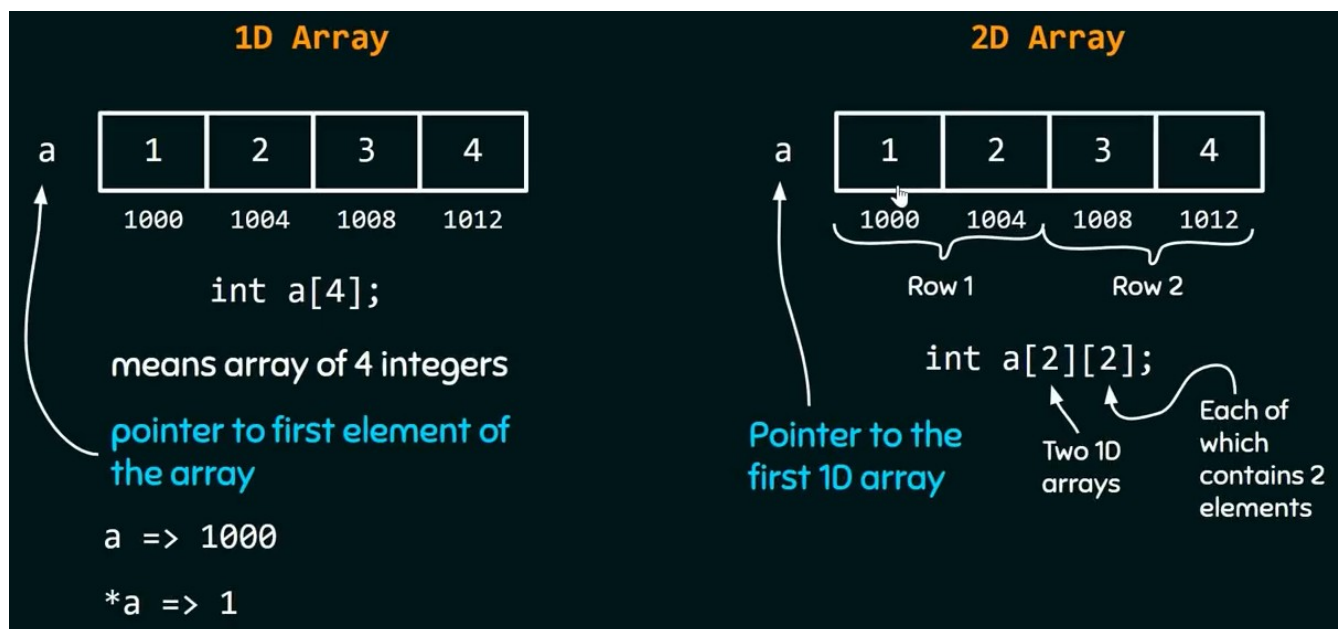


Column major order: Elements are stored column by column



***C/C++ follows row major order.

***Array & Multi-dimensional Array in terms of pointer:

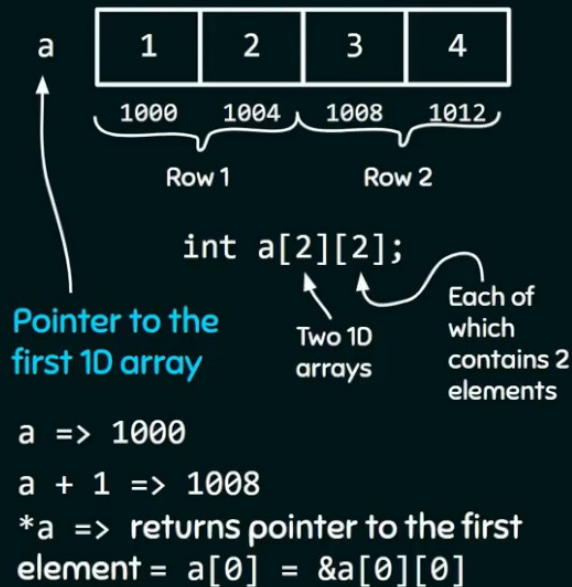


int a[2][2]; Means 'a' is 2D array, it has 2 one dimensional array, each array has 2 elements.

In **1D** array, array 'a' points to the first element of the array, But in multi-dimensional(**2D, 3D, 4D**) array array 'a' points to the 1st row(or 1st 1D array) of the array, not the 1st element, but points on all elements of the 1st row.

**See lec-118(Processing the multidimensional array elements)

2D Array



**a => *(*a) => *(*a+0))

=> *(&a[0][0]) => a[0][0] => 1

*(a+1) => *(pointer to the 2nd 1D array)

=> pointer to the 1st element of 2nd 1D array

** (a+1) => ** (pointer to the 2nd 1D array)

=> *(pointer to the 1st element of 2nd 1D array) => 3

*(a+1)+1 => pointer to the 1st element of 2nd 1D array + 1 => pointer to the 2nd element of 2nd 1D array

((a+1)+1) => *pointer to the 2nd element of 2nd 1D array => 4 => a[1][1]



This is 2D array. if we write 'a' it points to the 1st 1D array.

`a=> 1000` (pointer to 1st 1D array)

`a+1=> 1008` (pointer to 2nd 1D array)

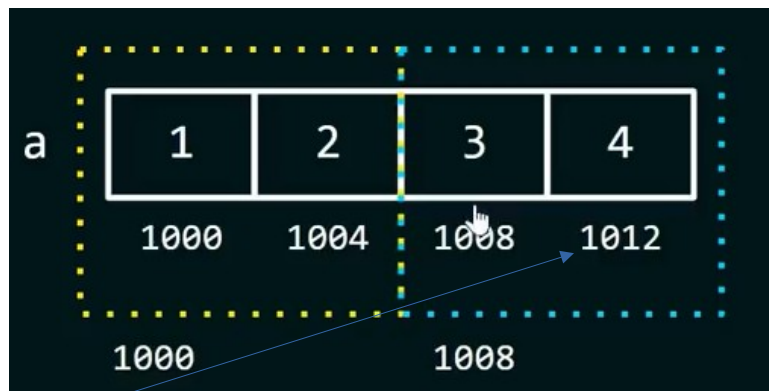
`*a=> *(pointer to 1st 1D array)` // putting '*' means get inside 1D array, and get 1000 as address as the 1st element of the array. So we can write `>> *a=> pointer to the 1st element of the 1st 1D array.` Contains address of 1st element of the 1st 1D array.

Or we can write `*a=> *(a+0) => a[0] => &a[0][0];`

`**a=> value of the 1st element of the 1st 1D array.` // dereferencing, get inside and grab the value of that address. So `**a=> 1`

`*(a+1)=> 1008`(pointer to the 1st element of the 2nd 1D array.)

`**a=> 1`



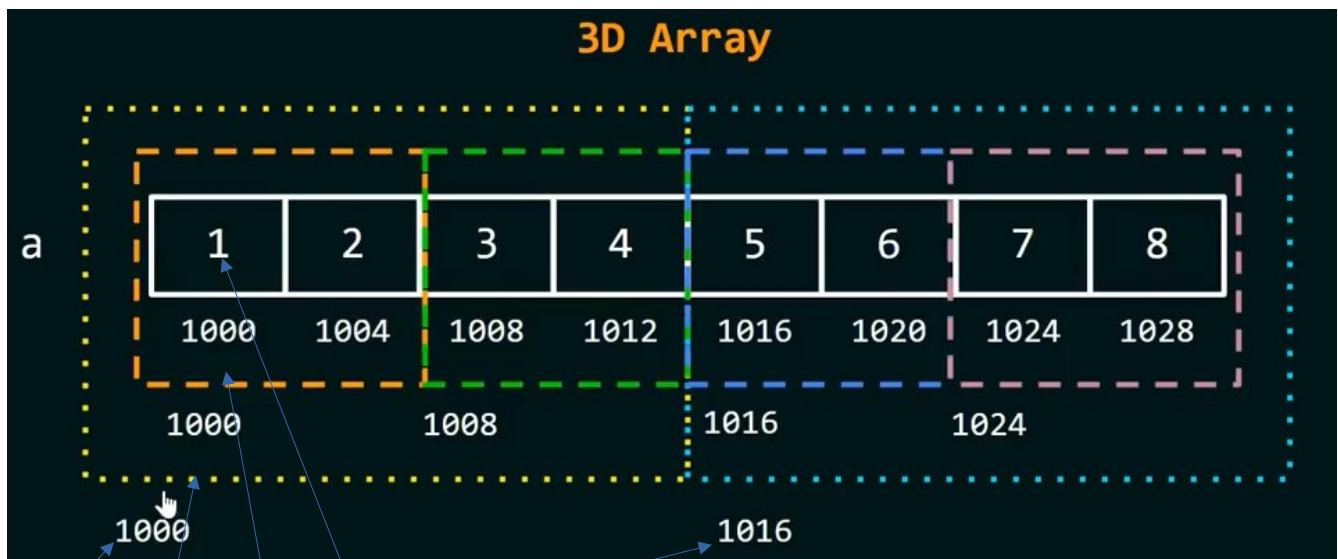
`*(a+1)+1=> 1012`

`*(a+1)=> 2`

`*(a+1)+1` or `**((a+1)+1)=> 4;`

`a[2][2][2]=>` means 'a' is 3D array. It has 2 two dimensional array. Both 2D array has 2 one dimensional array. Both 1D array has 2 elements.

3D array:



`a=>1000;`

`a+1=> 1016;`

`*a=>1000;`

`**a=>1000`

`***a=1;`

Same logic for 3D as 2D array.

-----X-----

*Manually point to entire elements of an array.

```
#include <stdio.h>

int main() {
    int a[5] = {1, 2, 3, 4, 5};
    int *p = a;
    printf("%d", *p);
    return 0;
}
```

← Pointer to the first element of array.

OUTPUT: 1

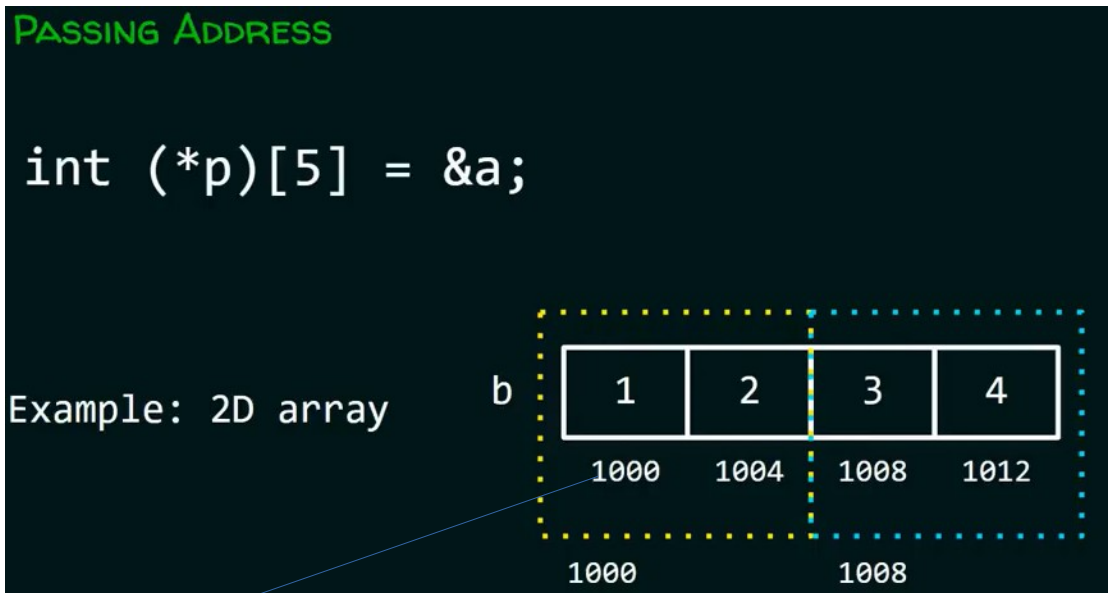
Replace `"int *p=a;"` with

`int (*p)[5]=&a; // means pointer to the whole array of 5 elements.`

Why using `"&a"` instead of `"a"`?

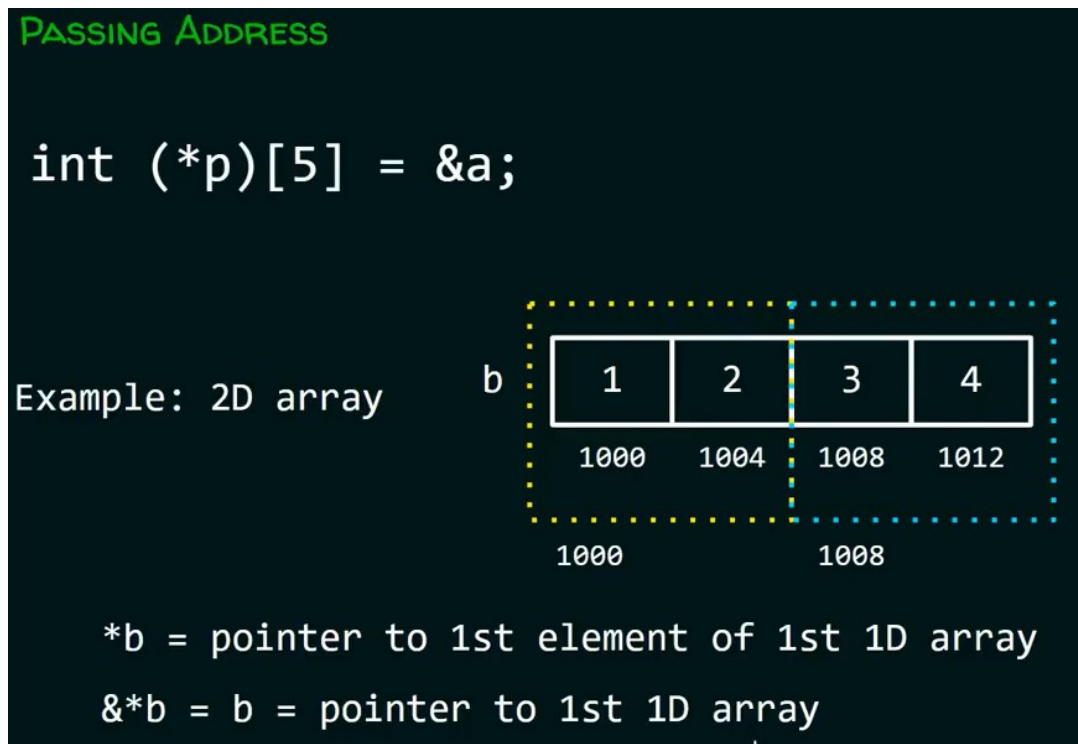
ans:

Logic's are same as 2D and 3D array. "P" points to the whole array. When writing "*p" it get inside and points to the 1st element of the array.



So "**p*" == *a*; // as we know "*a*" points to 1st element of the array.

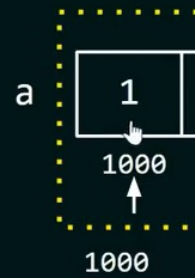
To come outside and points to whole array we need the "&" (as address(1000, outer address) of the address(1000, inner address) of 1st element)



PRINTING THE VALUE

```
int (*p)[5] = &a;  
printf("%d", **p);
```

OUTPUT: 1



-----X-----

Example:

What is the output of the following program:

```
#include <stdio.h>  
int main()  
{  
    int a[][3] = {1, 2, 3, 4, 5, 6};  
    int (*ptr)[3] = a;  
    printf("%d %d ", (*ptr)[1], (*ptr)[2]);  
    ++ptr;  
    printf("%d %d", (*ptr)[1], (*ptr)[2]);  
    return 0;  
}
```

- a) 2 3 5 6
- b) 2 3 4 5
- c) 4 5 0 0
- d) None of the above

$(*ptr)[1] = *((*ptr) + 1)$


```

#include <stdio.h>
int main()
{
    int a[][3] = {1, 2, 3, 4, 5, 6};
    int (*ptr)[3] = a;
    printf("%d %d ", (*ptr)[1], (*ptr)[2]);
    ++ptr;
    printf("%d %d", (*ptr)[1], (*ptr)[2]);
    return 0;
}

```

ptr is pointer to first 1D array

*ptr is pointer to first element of first 1D array

(*ptr)[1] = *((*ptr) + 1)

a	1	2	3	4	5	6
---	---	---	---	---	---	---

Row 1 Row 2

Ans: 2 3 5 6

-----X-----

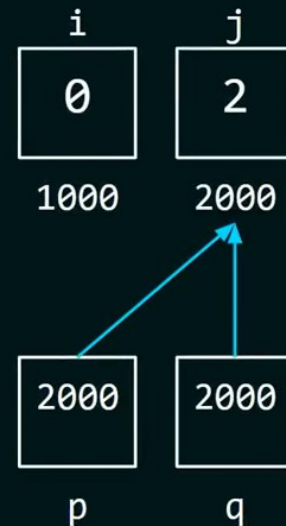
What is the output of the following C program:

```

void f(int *p, int *q)
{
    p = q;
    *p = 2;
}

int i = 0, j = 1;
int main() {
    f(&i, &j);
    printf("%d %d\n", i, j);
    return 0;
}

```



-----X-----

What is printed by the following C program:

```
int f(int x, int *py, int **ppz)
{
    int y, z;
    **ppz += 1;
    z = **ppz;
    *py += 2;
    y = *py;
    x += 3;
    return x + y + z;
}

void main()
{
    int c, *b, **a;
    c = 4, b = &c, a = &b;
    printf( "%d", f(c,b,a));
}
```

- a) 18
- b) 19
- c) 21
- d) 22

[GATE 2004]

Ans: 19 // see lec-125 no lecture(pointer problem 7)

Array of pointers:

TWO DIMENSIONAL ARRAY

```
char fruits[][12] = {"2 Oranges", "2 Apples", "3 Bananas", "1 Pineapple"};
```

2		O	r	a	n	g	e	s	\0	\0	\0
2		A	p	p	l	e	s	\0	\0	\0	\0
3		B	a	n	a	n	a	s	\0	\0	\0
1		P	i	n	e	a	p	p	l	e	\0

A LOT OF SPACE IS SIMPLY WASTED.

Wastage of memory cause of null character. Instead of it we use array of pointers.

See lec 142. Array of string

ARRAY OF POINTERS

```
char *fruits[] = {"2 Oranges", "2 Apples", "3 Bananas", "1 Pineapple"};
```



Void Pointer:

Void pointer is a pointer which has no associated data type with it.

It can point to any data of any data type and can be typecasted to any type.

EXAMPLE:

```
int main()
{
    int n = 10;
    void *ptr = &n;

    printf("%d", *(int*)ptr);
    return 0;
}
```

We can't dereference void pointer directly. We have to typecast to use it. See **printf** part on above example.

Use of void pointer(why we use): malloc and calloc function returns void pointer. Due to this reason, they can allocate memory for any data type.

NULL pointer: Null pointer is pointer which does not point to any memory location. It represents an invalid memory location.

```
int main() {  
    int *ptr = NULL;  
    return 0;  
}
```

Uses:

* It is used to initialize a pointer when that pointer isn't assigned any valid memory address yet.

* use when a memory is released.

*** The value of NULL is 0. This '0' is not same as integer value '0'. We can use '0' or 'NULL'. Both same.

** Size of NULL pointer may depends on platform. Usually 8 byte.

Dangling Pointer: A pointer which points to non existing memory location. Suppose, a memory allocate with pointer. When that memory freed. Pointer which is pointing to that memory, is pointing to deallocated memory. This type of pointer called dangling pointer. To avoid this set pointer to NULL after freed memory.

```
int main()  
{  
    int *ptr = (int *)malloc(sizeof(int));  
    ...  
    ...  
    free(ptr);  
    ptr = NULL;  
    return 0;  
}
```

Now, ptr is no more dangling.

Wild Pointer: Wild pointers are also known as uninitialized pointers.

Wild pointers are also known as uninitialized pointers.

These pointers usually point to some arbitrary memory location and may cause a program to crash or misbehave.

EXAMPLE:

```
int main()  
{  
    int *p;  
    *p = 10;  
    return 0;  
}
```

Here pointer *P* is pointing to any arbitrary memory. That memory may occupy another program. Accessing that memory may cause crash.

Pointer to Pointer(Double Pointer):

```
int main()
{
    int val = 35;
    int* ptr;
    ptr = &val;
    printf("%d\n", *ptr);

    change_val(ptr);
    printf("%d", *ptr);
    return 0;
}
```

```
void change_val(int* ptr)
{
    int val2 = 46;
    ptr = &val2;
}
```

Output:
35
35

Assume that the address of the val2 is 2000.

Here above a copy of a pointer is passing(pass by value). That's why value not changing.

If we pass reference

```
int main()
{
    int val = 35;
    int* ptr;
    ptr = &val;
    printf("%d\n", *ptr);

    change_val(&ptr);
    printf("%d", *ptr);
    return 0;
}
```

```
void change_val(int** ptr1)
{
    int val2 = 46;
    *ptr1 = &val2;
}
```

Output:
35
46

-----X-----

String:

**String literals are stored as array of characters. It is always ended with '\0'(null character). which indicates the end of the string.*

In C, Compiler treats a string literal as a pointer to the first character.



So to the printf or scanf, we are passing a pointer to the first character of a string literal.

Both printf and scanf functions expects a character pointer (char *) as an argument.

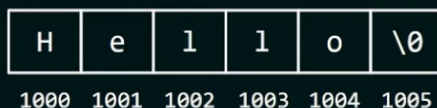
```
printf("Earth");
```

Passing "Earth" is equivalent to passing the pointer to letter 'E'

```
char *ptr = "Hello"
```

As writing "Hello" is equivalent to writing the pointer to the first character. Therefore, we can subscript it to get some character of the string literal

"Hello"[1] is equivalent to pointer to 'H'[1]



pointer to 'H'[1] = *(pointer to 'H' + 1)

pointer to 'H'[1] = *(1000 + 1) = *(1001) = 'e'

POINT TO BE NOTED

String literal cannot be modified. It causes undefined behaviour.

```
char *ptr = "Hello";  
ptr = 'M'; ← Not Allowed.
```

String literals are also known as string constants.
They have been allocated a read only memory. So we cannot alter them.

But character pointer itself has been allocated read-write memory. So the same pointer can point to some other string literal.

"%m.ns" is used to print just a part of the string where **n** is the number of characters to be displayed and **m** denotes the size of the field within which the string will be displayed.

```
char *ptr = "Hello World!";  
printf("%.5s", ptr);  
printf("%6.5s", ptr);
```

OUTPUT: Hello
Hello



Puts(): puts() function used to write string to the output string. This automatically add a new line after string.

Scanf():

scanf("%s", a);

* here 'a' is treated as a pointer of the first element of the array. So no need to put '&' like int variable.

* it store all character except white space. After white space it ignore all character. "you are fool".

Here scanf will store 'you'

gets(): This is used to read entire line as input.

*** scanf() & gets() both function have no way to detect when the character array is full. May cause undefined behavior, may lead to buffer overflow error. Program may crash.

Although, `scanf()` has the way to set the limit for the number of characters to be stored in the character array.

By using `%ns`, where `n` indicates the number of characters allowed to store in the character array.

```
char a[10];  
printf("Enter the string:\n");  
scanf("%9s", a);  
printf("%s", a);
```

But, unfortunately, `gets()` is still **UNSAFE**.

It will try to write the characters beyond the memory allocated to the character array which is **unsafe** because it will simply overwrite the memory beyond the memory allocated to the character array.

getchar(): this function is used to read 1 character at a time from the user input. It returns integer value of that inputted character ASCII value.

Putchar(): `putchar()` take int value as argument, and print character of the corresponding int.

```
int a=65;
```

```
putchar(a); /// output>> "A"
```

****strcpy()***: `strcpy` defined on `<string.h>` header file. It returns the pointer of the 1st character of copied string.

Prototype: `char* strcpy(char* destination, const char* source)`

It isn't modified. That is why it is constant.

`strcpy` is used to copy a string pointed by source (including NULL character) to the destination (character array)

Example:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[10] = "Hello";
    char str2[10];

    printf("%s\n", strcpy(str2, str1));
    printf("%s", str2);
    return 0;
}
```

strcpy returns the pointer to the first character of the string which is copied in the destination. Hence if we use %s, then whole string will be printed on the screen.

Output: Hello

output: both will print "Hello"

Caution: `strcpy(str2, str1);`

Here str2 will copy to str1. If str2 size is greater than str1, then there will be undefined behavior. No way to check whether str2 will fit to str1. In that case we should use **strncpy()**.

Strncpy():

```
strncpy(destination, source, sizeof(destination));
```

Example:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[6] = "Hello";
    char str2[4];
    strncpy(str2, str1, sizeof(str2));
    printf("%s", str2);
    return 0;
}
```

OUTPUT:
Hell

Here only character will copy according to size of destination(str1).

*** **strncpy** never add null character(\0) in the end. It is necessary to add.

```
str2[sizeof(str2)-1]='\0';
```

strncpy will leave the string in str2 (destination) without a terminating NULL character, If the size of str1 (source) is equal to or greater than the size of str2 (destination).

strlen(): it returns the size of string.

* it doesn't count null character.

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100] = "Hello World";
    printf("%d", strlen(str));
    return 0;
}
```

OUTPUT: 11

It calculates the length of the string and not the length of the array.

strcat(): string concatenation function.

```
char str1[100]="Hello";
char str2[100]=" World";
strcat(str1, str2);
printf("%s\n", str1);
```

Here str2 will add at the end of str1.

** str1 size have to enough to concatenate str2. Otherwise undefined behavior may occur. So we can use **strncat**.

strncat is the **safer version** of strcat.

It appends the limited number of characters specified by the third argument passed to it.



Note: strncat automatically adds NULL character at the end of the resultant string.

```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char str1[5], str2[100];
    strcpy(str1, "He");
    strcpy(str2, "llo!");
    strncat(str1, str2, sizeof(str1) - strlen(str1) - 1);
    printf("%s", str1);
    return 0;
}
```

Size of the array

Size of the string

Creating room
for NULL
character

Ans: "Hell" // $5-2-1=2$ only 2 character will concatenate

* strncat add '\0' at the end of concatenating string.

***strcmp():**

STRCMP (STRING COMPARISON) FUNCTION

Prototype: `int strcmp(const char* s1, const char* s2);`

★ Compares two strings s1 and s2

★ Returns value
Less than 0, if $s1 < s2$
Greater than 0, if $s1 > s2$
Equal to 0, if $s1 == s2$

ATTENTION!!!

IN ASCII CHARACTER SET

- ★ All upper case letters are less than all the lower case letters (Upper case letters have ASCII codes between 65 and 90 and Lower case letters have ASCII codes between 97 and 122)
- ★ Digits are less than letters (0 – 9 digits have ASCII codes between 48 and 57)
- ★ Spaces are less than all printing characters (Space character has the value 32 in ASCII set)

See Lec 143-147 (String problems)

Function Pointers: Function pointers are like normal pointers but they have capability to point to a function. Function are set of instructions which are stored in a memory. And they have memory location. Pointer point to that function location.

*** `int *ptr[10];` //10 pointer pointing to array elements. See lec 148 (function pointers in C)

so we can write:

int (*ptr)[10]; // **ptr** is a pointer which pointing to an array(of 1st element), first brackets used caused '[' precedence is higher than '*'.

Declaring a function:

int (*ptr)(int, int)=&add; //here '**add**' is a function not written here. It means **ptr** is a pointer which point to function which have 2 arguments. And returns an integer. After assigning '&add' it will point to that function.

Or

int (*ptr)(int, int);

ptr=&add;

2 way we can write a function and call function with pointers.

```
int add(int a, int b)
{
    return a+b;
}
```

```
int main()
{
    int result;
    int (*ptr)(int, int) = &add;
    result = *ptr(10, 20);
    printf("%d", result);
}
```

```
int add(int a, int b)
{
    return a+b;
}
```

```
int main()
{
    int result;
    int (*ptr)(int, int) = add;
    result = ptr(10, 20);
    printf("%d", result);
}
```

differences: if we use '&' we need to put '*' as dereference operator. Otherwise don't need both.

Replace: result=(*ptr)(10,20);

Application of function pointers:

calculator: Where we can use if/switch case. But we can use function pointers. **See lec 149:**
(application of function pointer)

-----X-----

Structure: Structure is a user defined data type that can be used to group elements of different types into a single type.

Structure Tag: Structure tag is used to identify a particular type of structure.

NEED OF CREATING A TYPE

```
struct {
    char *name;
    int age;
    int salary;
} emp1, emp2;

int manager()
{
    struct {
        char *name;
        int age;
        int salary;
    } manager;

    ...
}
```

```
struct employee {
    char *name;
    int age;
    int salary;
};

int manager()
{
    struct employee manager;
    ...
}

int main()
{
    struct employee emp1;
    struct employee emp2;
    ...
}
```

Structure tag
↓

Typedef: Typedef gives freedom to the user by allowing them to create their own type.

Syntax: typedef existing_data_type new_data_type

```
struct car {  
    char engine[50];  
    char fuel_type[10];  
    int fuel_tank_cap;  
    int seating_cap;  
    float city_mileage;  
};  
  
int main() {  
    struct car c1;  
}
```

We can write this with typedef... mentioned below

```
typedef struct car { ← Old Type  
    char engine[50];  
    char fuel_type[10];  
    int fuel_tank_cap;  
    int seating_cap;  
    float city_mileage;  
}car; ← New Type
```

```
typedef struct car {  
    char engine[50];  
    char fuel_type[10];  
    int fuel_tank_cap;  
    int seating_cap;  
    float city_mileage;  
}car;  
  
int main() {  
    car c1;  
}
```

car becomes a new data type.

Structure Padding:

When an object of some structure type is declared then some contiguous block of memory will be allocated to structure members.

For example:

```
struct abc {  
    char a;  
    char b;  
    int c;  
} var;
```

Here 'var' is an object. Here we can assume structure 'abc' should have 6 byte. But wrong.

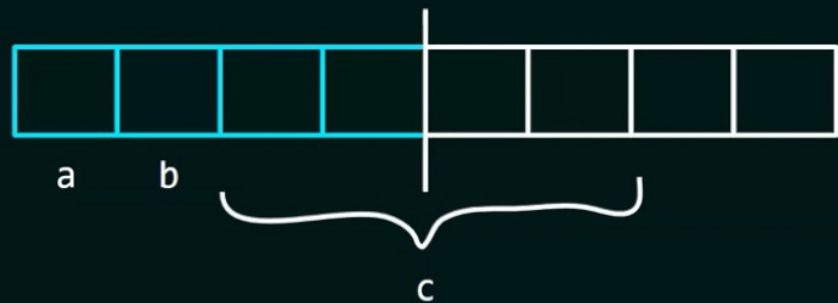
There is a concept 'Structure padding'

Processor doesn't read 1 byte at a time from memory. It read 1 word at a time.

This means: If we have 32 bit processor, it means it can access 4 byte(32 bit) at a time, which means word size is 4 byte. For 64 bit processor word size is 8 byte.

```
struct abc {  
    char a; //1 byte  
    char b; //1 byte  
    int c;  //4 bytes  
} var;
```

In 32 bit architecture



In 1 CPU cycle, char a, char b and 2 bytes of int c can be accessed. There is no problem with char a and char b but...

12

```

struct abc {
    char a; //1 byte
    char b; //1 byte
    int c;  //4 bytes
} var;

```

In 32 bit architecture

Whenever we want the value stored in variable c, 2 cycles are required to access the contents of variable c. In first cycle, 1st 2 bytes can be accessed and in 2nd cycle, last 2 bytes.

Accessing int c, required 2 cpu cycle. Wastage of cpu cycle. Using padding concept, empty room will be created here.

```

struct abc {
    char a; //1 byte
    char b; //1 byte
    int c;  //4 bytes
} var;

```

In 32 bit architecture

Total = 1 byte + 1 byte + 2 bytes + 4 bytes = 8 bytes

Order of member may effect the size of an object...

```

struct abc {
    char a;
    int b;
    char c;
};

```

For a 32 bit architecture

3 words accessed = 12 bytes

```

int main()
{
    struct abc var;
    printf("%d", sizeof(var));
}

```

Union:

UNIONS

Union is a user defined data type but unlike structures, union members share same memory location.

Example:

```
struct abc {  
    int a;  
    char b;  
};
```

a's address = 6295624
b's address = 6295628

```
union abc {  
    int a;  
    char b;  
};
```

a's address = 6295616
b's address = 6295616

Size of the union is taken by size of the largest member of the union.

See size diff bet structure & union

```
typedef union {  
    int a;  
    char b;  
    double c; }  
data;
```

Size = 8 bytes

```
int main()  
{  
    data arr[10];    Size = 80 bytes  
    arr[0].a = 10;  
    arr[1].b = 'a';  
    arr[2].c = 10.178;  
    //and so on  
    return 0;  
}
```

```
typedef struct {  
    int a;  
    char b;  
    double c; }  
data;
```

Size = 13 bytes

```
int main()  
{  
    data arr[10];    Size = 130 bytes  
    arr[0].a = 10;  
    arr[1].b = 'a';  
    arr[2].c = 10.178;  
    //and so on  
    return 0;  
}
```